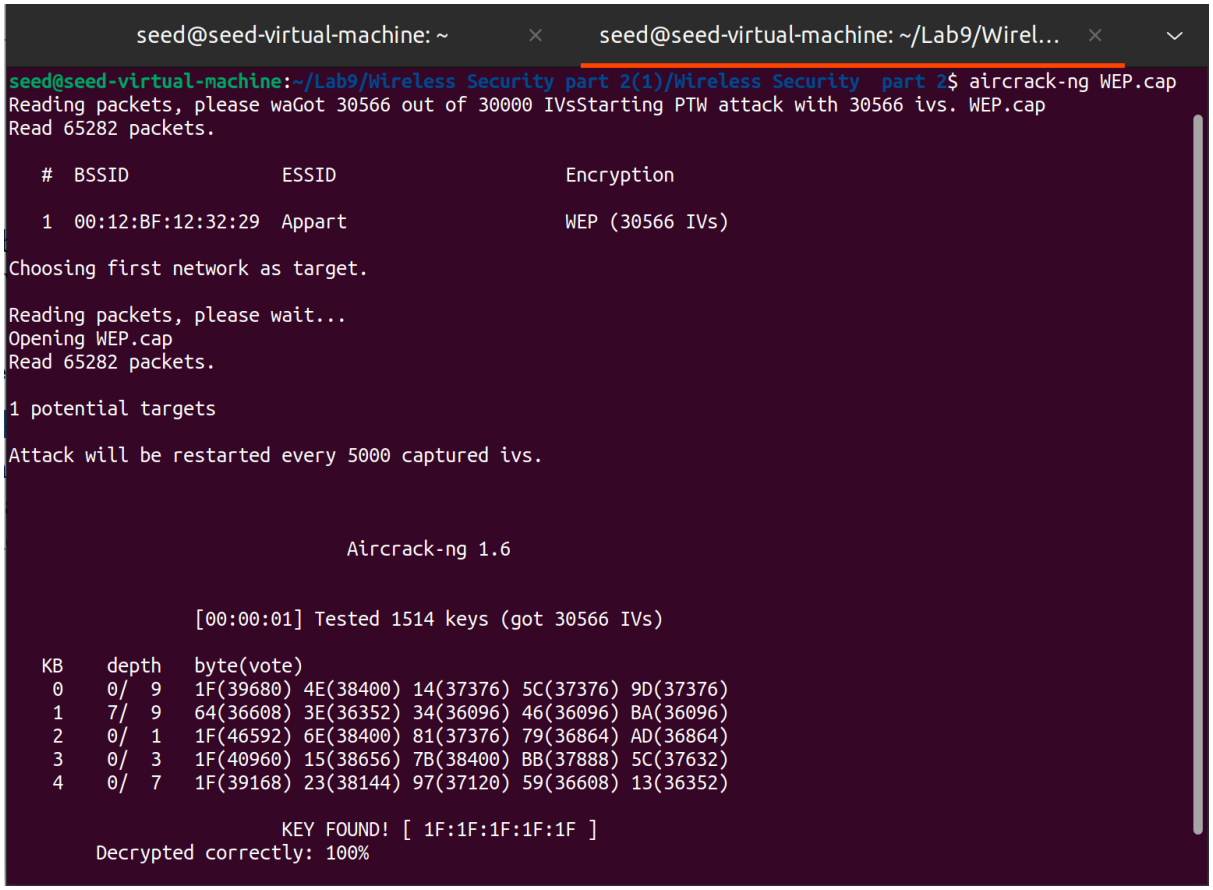
50.020 Network Security

Siddharth Ganesh
1006407

Lab 9 Report

Task 1: Cracking the WEP Password

For this task, I installed Aircrack-ng and used the aircrack-ng WEP.cap file to obtain the cracked key to be 1F1F1F1F1F. I ran this command to visualise and verify the working of the Aircrack-ng package, even though this key has already been provided:



Task 2: Cracking the WEP Packet

Step 2: Implement the RC4 Algorithm

Here is the code I used for this task in the skeleton.py file:

```python
import binascii
import struct

def KSA(key):
    S = list(range(256))
    # Add KSA implementation Here
    j = 0
    for i in range(256):
        j = (j + S[i] + key[i % len(key)]) % 256
        S[i], S[j] = S[j], S[i]
    return S

def PRGA(S):
    # Add PRGA implementation here
    i = 0
    j = 0
    while True:
        i = (i + 1) % 256
        j = (j + S[i]) % 256
        S[i], S[j] = S[j], S[i]
        yield S[(S[i] + S[j]) % 256]

def RC4(key):
    S = KSA(key)
    return PRGA(S)

def decrypt(key, ciphertext):
    key = binascii.unhexlify(key)
    ciphertext = binascii.unhexlify(ciphertext)
    keystream = RC4(key) #generating keystream using the RC4 algo

    #process to crack ciphertext
    payload = ""
    for i in ciphertext:
        payload += ('{:02X}'.format(i ^ next(keystream)))

    return payload

if __name__ == '__main__':
    # RC4 algorithm please refer to http://en.wikipedia.org/wiki/RC4

    ## key = a list of integer, each integer 8 bits (0 ~ 255)
    ## ciphertext = a list of integer, each integer 8 bits (0 ~ 255)
    ## binascii.unhexlify() is a useful function to convert from Hex string to integer list

    ## Cracking the ciphertext

    #     Several test cases: (to test RC4 implementation only)
    #     1. key = '1A2B3C', cipertext = '00112233' -> plaintext = '0F6D13BC'
    #     2. key = '000000', cipertext = '00112233' -> plaintext = 'DE09AB72'
    #     3. key = '012345', cipertext = '00112233' -> plaintext = '6F914F8F'

    plaintext = decrypt('1A2B3C', '00112233')
    assert plaintext == '0F6D13BC'
    print(f"plaintext == {plaintext}; Test Case 1 passed")

    plaintext = decrypt('000000', '00112233')
    assert plaintext == 'DE09AB72'
    print(f"plaintext == {plaintext}; Test Case 2 passed")

    plaintext = decrypt('012345', '00112233')
    assert plaintext == '6F914F8F'
    print(f"plaintext == {plaintext}; Test Case 3 passed")
```

The code implements the RC4 stream cipher algorithm, which is used to encrypt or decrypt data using a symmetric key. The implementation consists of these functions:

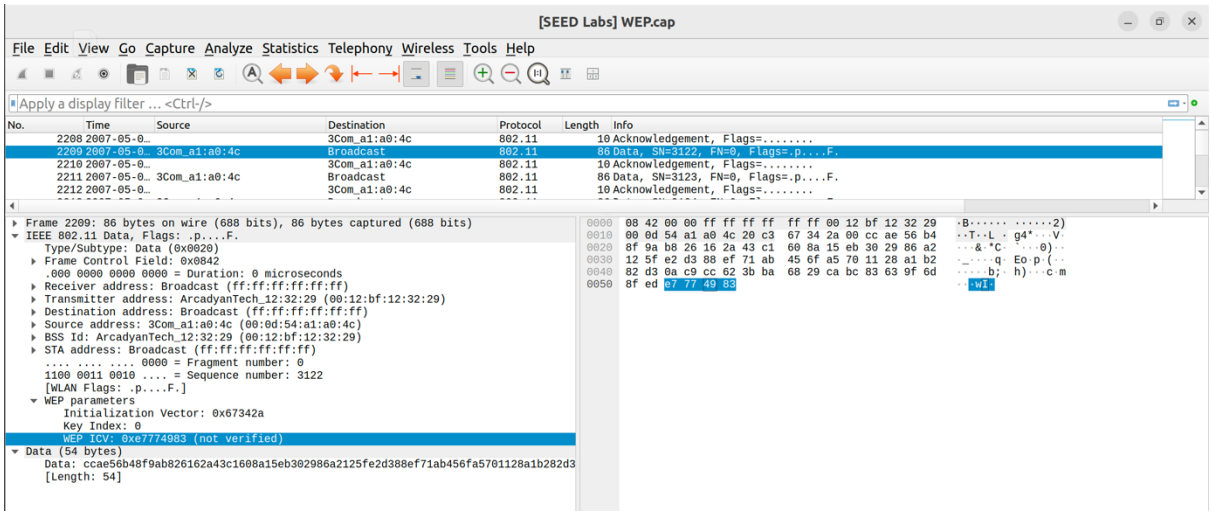
- Key-Scheduling Algorithm (KSA): Initializes the state array (S) based on the input key.
- Pseudo-Random Generation Algorithm (PRGA): Generates the keystream based on the initialized state array.
- RC4 Function: Combines KSA and PRGA to produce the keystream.
- Decrypt Function: Uses the keystream to decrypt a given ciphertext. I wrote this in a separate function to encapsulate the decryption process. The decrypt function applies the RC4 keystream to a given ciphertext to produce the plaintext.

The program successfully decrypts the given ciphertexts for the test cases:

```
seed@seed-virtual-machine:~/Lab9/Wireless Security part 2(1)/Wireless Security  part 2$ python3 skeleton.py
plaintext == 0F6D13BC; Test Case 1 passed
plaintext == DE09AB72; Test Case 2 passed
plaintext == 6F914F8F; Test Case 3 passed
seed@seed-virtual-machine:~/Lab9/Wireless Security part 2(1)/Wireless Security  part 2$
```

Step 3: Verify Your Results

In this task, we attempt to crack packets from WEP.cap. I selected the packet with SN = 3122.



For this packet, the details are as follows:

- Initialisation Vector: 67342a
- WEP ICV: e7774983
- Data: ccae56b48f9ab826162a43c1608a15eb302986a2125fe2d388ef71ab456fa5701128a1b282d30ac9cc623bba6829cabc83639f6d8fed

From Task 1, the key is 1F1F1F1F1F.

With this information, we can complete the skeleton.py file, as below:

```python
import binascii
import struct

def KSA(key):
    S = list(range(256))
    # Add KSA implementation Here
    j = 0
    for i in range(256):
        j = (j + S[i] + key[i % len(key)]) % 256
        S[i], S[j] = S[j], S[i]
    return S

def PRGA(S):
    # Add PRGA implementation here
    i = 0
    j = 0
    while True:
        i = (i + 1) % 256
        j = (j + S[i]) % 256
        S[i], S[j] = S[j], S[i]
        yield S[(S[i] + S[j]) % 256]

def RC4(key):
    S = KSA(key)
    return PRGA(S)

def decrypt(key, ciphertext):
    key = binascii.unhexlify(key)
    ciphertext = binascii.unhexlify(ciphertext)
    keystream = RC4(key) #generating keystream using the RC4 algo

    #process to crack ciphertext
    payload = ""
    for i in ciphertext:
        payload += ('{:02X}'.format(i ^ next(keystream)))

    return payload

if __name__ == '__main__':
    key = '1F1F1F1F1F'
    iv = '67342a'
    ivKey = binascii.unhexlify(iv + key)

    encrypted_msg = 'ccae56b48f9ab826162a43c1608a15eb302986a2125fe2d388ef71ab456fa5701128a1b282d30ac9cc623bba6829cabc83639f6d8fed'
    encrypted_icv = 'e7774983'
    ciphertext = binascii.unhexlify(encrypted_msg + encrypted_icv)

    keystream = RC4(ivKey) #generate keystream using the RC4 algorithm

    payload = ""
    for i in ciphertext:
        payload += ('{:02X}'.format(i ^ next(keystream)))
    print(f"payload: {payload}")

    decrypted_icv = payload[-8:]
    print(f"decrypted ICV: {decrypted_icv}")

    decrypted_msg = payload[:-8]

    crcle = binascii.crc32(bytes.fromhex(decrypted_msg)) & 0xffffffff
    crc = struct.pack('<L', crcle) # converting little endian to big endian
    crc = binascii.hexlify(crc).decode().upper()
    print(f"checksum: {crc}")

    assert decrypted_icv == crc
    print("Checksum verification complete")
```

Running skeleton.py now provides the payload, decrypted ICV and the checksum as below:

```
seed@seed-virtual-machine:~/Lab9/Wireless Security part 2(1)/Wireless Security  part 2$ python3 skeleton.py
payload: AAAA0300000008060001080006040001000EA66BFB69AC10000100000000000AC1000F000000000000000000000000000000000000006B8FE49D
decrypted ICV: 6B8FE49D
checksum: 6B8FE49D
Checksum verification complete
seed@seed-virtual-machine:~/Lab9/Wireless Security part 2(1)/Wireless Security  part 2$
```

Assignment

1. *Read the instructions above and use the skeleton python code to finish all the tasks. You may crack any broadcast WEP packet, e.g., SN=2000, for the lab.*

   All the tasks have been completed according to the instructions in the lab sheet. The details of each task have been provided across the previous pages of this report.

2. *Demonstrate the following in the report*
   a. *Justify the correctness of your implementation of the RC4 algorithm*

      The correctness of the RC4 implementation is justified by its adherence to the algorithm's standard design, as described in reputable sources such as the RC4 Wikipedia page. The Key-Scheduling Algorithm (KSA) properly initializes the state array S using the input key, ensuring that the resulting state array reflects the key's influence through a series of swaps based on the key values. The Pseudo-Random Generation Algorithm (PRGA) generates a keystream by further permuting the state array dynamically and yielding output bytes derived from the current state indices. The implementation was tested against known test cases, where specific keys and ciphertexts produced the expected plaintext outputs, as shown in the output of Task 2 Step 2. This validation demonstrates that the functions accurately generate the keystream and apply XOR operations for encryption and decryption, as per the RC4 specification.

```
seed@seed-virtual-machine:~/Lab9/Wireless Security part 2(1)/Wireless Security  part 2$ python3 skeleton.py
plaintext == 0F6D13BC; Test Case 1 passed
plaintext == DE09AB72; Test Case 2 passed
plaintext == 6F914F8F; Test Case 3 passed
seed@seed-virtual-machine:~/Lab9/Wireless Security part 2(1)/Wireless Security  part 2$
```

   b. *The cracked payload and ICV of one broadcast packet*

      Cracked payload: AAAA0300000008060001080006040001000EA66BFB69AC10000100000000000AC1000F000000000000000000000000000000000000006B8FE49D

      ICV: 6B8FE49D

      The above information was obtained from the output of running the skeleton.py file:

```
seed@seed-virtual-machine:~/Lab9/Wireless Security part 2(1)/Wireless Security  part 2$ python3 skeleton.py
payload: AAAA0300000008060001080006040001000EA66BFB69AC10000100000000000AC1000F000000000000000000000000000000000000006B8FE49D
decrypted ICV: 6B8FE49D
checksum: 6B8FE49D
Checksum verification complete
seed@seed-virtual-machine:~/Lab9/Wireless Security part 2(1)/Wireless Security  part 2$
```